At the time the system went live, there were 81 known issues with the software and no load-tests had been run. There were no provisions for a backup system either. While the gap of 10 months between the time dispatchers were first trained to use the software and when it was deployed played its role in the disaster, the software had three primary flaws that immediately caused the failure:

Edge case data
The software system did not function well when given invalid or incomplete data regarding the positions and statuses of ambulances.

Interface issues
The deployed system had a wide variety of errors in different parts of the user interface. For example, parts of the MDT terminal screens had black spots, thus preventing ambulance operators from getting all information needed.

Memory Leak
The root cause of the main breakdown of the system, however, was a memory leak in a small portion of code. This defect retained memory that held information on the file server even after it was no longer needed. As with any memory leak, the memory eventually filled up and caused the system to fail.

Why did these things happen?
In each of the incidents above, one interesting fact to note is how a majority of them were not really an issue of coding quality or machine fault but how they were all to some degree or another caused by poor design or bad coding standards or even abstract and vague requirements.

While delivery of a finished application in due time is crucial, sacrificing design time or ignoring planning practices can ultimately result in much higher losses than expected. As is evident from the case studies above, the simplest of errors either in development, planning or design can compound into unforeseen losses.

The essence of this chapter thus, is to impress upon the reader the importance of the processes that go into planning the development of an application, designing it to the clearest granularity of detail, and only when you have defined both of these clearly should you begin to develop it. d